

NFC Medicine Authenticity System

Production-Hardening Changes • What changed and what's left before deployment

This document summarizes the work done to turn the localhost NFC anti-counterfeiting demo (RPi4 + PN532 writing tags, Flask/Postgres backend) into an architecture that could actually verify medicine authenticity for a real user, plus the concrete steps still needed before it's deployment- and demo-ready.

1. Starting Point

Before this work, the project was a working localhost demo with several real gaps for anything beyond a lab test:

- **No consumer verification flow at all.** There was no webpage and no endpoint for a customer to check whether a scanned medicine tag was genuine.
- **GET /api/products was fully unauthenticated** and returned every record in the database.
- **The nonce field was stored but never checked** — replay protection was really just a 30-second signature timestamp window.
- **Key-derivation material (key_chars) was stored in plaintext** in Postgres.
- **The custom “paper cipher”** (a bit-shift/inversion block cipher matching the accompanying research paper, Section III-B/III-C) was the only crypto protecting stored data — real, but cryptographically weak by production standards.
- The working Raspberry Pi script was misfiled as `backend/Pasted text.txt`; the real `pi_app.py` at the repo root was empty.
- The project was **not a git repository** — no version history, nothing reversible.

2. What Changed

2.1 Version control

- Initialized git and committed the original state first, so every change made since is reversible via history.

2.2 Consumer verification — the core missing piece

- New GET `/api/verify/<tag_uid_hash>`: public, rate-limited endpoint. Looks up the tag, decrypts it **server-side**, and returns only a status (*authentic / expired / tampered / unknown*) plus safe display fields.
- The client never receives ciphertext, IVs, key material, or any secret — this is the architectural fix that makes a browser-based scan safe.
- Status precedence: `tampered > expired > authentic`, so a tampered-but-not-yet-expired record is never shown as safe.

2.3 Cryptography: modern crypto added alongside the untouched paper cipher

- New `backend/crypto_paper.py`: the original cipher moved out byte-for-byte identical (regression-tested against the old behavior) — kept exactly as-is so the paper's results stay reproducible.

- New backend/crypto_modern.py: **AES-256-GCM** with a per-tag key derived via HKDF-SHA256 from a new AES_MASTER_KEY secret (distinct from the HMAC signing secret). No key material is ever transmitted or stored — both the Pi and the server derive the same key independently.
- A new crypto_version column tags each database row (paper_v1 or aes_gcm_v1) so both schemes coexist in one table without conflict.
- This directly fixes the plaintext-key-material problem: modern-path rows never populate the key_chars column at all.

2.4 Replay protection

- The nonce column now has a database-level **UNIQUE constraint**. A replayed write is rejected with 409 even if an attacker manages to produce a fresh, currently-valid signature over the same nonce — not just blocked by the 30-second timestamp window.

2.5 Access control

- The old anonymous GET /api/products (dumped every record) is **removed**.
- Replaced with GET /api/admin/products and GET /api/admin/keys/<hash>, both gated behind a new X-Admin-Key header, checked with constant-time comparison (backend/admin_auth.py).

2.6 Rate limiting & audit logging

- flask-limiter added on every route (verify: 20/min, product write: 30/min, admin list: 60/min, admin keys: 10/min).
- New audit_log table + backend/audit.py: every write and every verify attempt is recorded (event type, tag hash, result, source IP, user agent, timestamp) — used both for monitoring and as evidence for the attack-simulation deliverable.

2.7 Database migration

- backend/init_db.py changed from destructively dropping the table on every run to CREATE TABLE IF NOT EXISTS — safe to rerun.
- New backend/migrate.py: an additive, idempotent migration that adds crypto_version, makes key_chars nullable, adds the nonce UNIQUE constraint, and creates the audit_log table.

2.8 Consumer-facing webpage (new)

- New frontend/ folder: plain HTML/CSS/JS, no build step, served same-origin by Flask.
- verify.html calls the new verify endpoint and renders a large color-coded result: green Authentic, amber Expired, red Tampered, gray Unknown.
- index.html provides manual tag-ID entry for testing without physical hardware. A basic manifest.json makes it installable.

2.9 Raspberry Pi script

- Promoted from the misfiled backend/Pasted_text.txt into a proper pi/pi_app.py with its own pi/requirements.txt (hardware-specific dependencies no longer pollute the backend's requirements).
- BACKEND_URL and a new FRONTEND_VERIFY_BASE_URL moved from a hardcoded LAN IP into environment variables.
- Backend writes now use AES-256-GCM (crypto_version aes_gcm_v1) instead of transmitting key_chars.
- The on-tag paper-cipher write/read-back (NFC pages 4–7) is **completely untouched**.

- New: writes a standard **NDEF URI record** to pages 8+ pointing at the verify webpage, so tapping a tag opens the verification page natively on a phone. Falls back to generating a printable QR code if the NDEF record doesn't fit or the write fails.

2.10 Attack-simulation test suite (for your professor)

- New backend/`tests/`: a real pytest suite consolidating the old ad hoc scripts, plus dedicated attack simulations:
- **Replay** — verbatim replay, stale timestamp, and freshly-forged-signature-same-nonce, all rejected.
- **Clone** — writes without the shared secret rejected; a fabricated/blank tag UID correctly shows “unknown”.
- **Brute force** — random signatures against the write endpoint and random hash enumeration against verify, both rejected and eventually rate-limited (429).
- **MITM** — tampered ciphertext rejected before it reaches decryption/storage; a directly mutated database row is correctly flagged “tampered” on verify.
- Each test writes a structured evidence log; backend/`tests/report.py` cross-references it against the `audit_log` table and generates evidence/`report.md` — a ready-made results section.

2.11 Bug fixes found along the way

- `db.py / init_db.py` printed an emoji (“🐛”) on error, which **crashed with UnicodeEncodeError** on Windows' default console codepage — silently masking the real database error underneath every failure. Fixed to plain ASCII output.
- Fixed a database-connection leak: on an insert error (including the new nonce-uniqueness rejection), the connection was never closed or rolled back. Now every DB-touching route closes its connection in a `finally` block.
- Removed a stray, corrupted duplicate `requirements.txt` at the repo root and the empty stray root `pi_app.py`.

3. Known Limitations (Flagged, Not Silently Solved)

These were deliberately left as documented trade-offs rather than solved, either because they're genuine extra scope or because they're the correct subject of your next phase:

- **Physical tag cloning:** verification works by looking up a server-side record keyed on `tag_uid_hash`. It does not cryptographically prove the physical NFC chip is genuine — a cloned tag broadcasting a copied UID and copied NDEF link would still resolve to the real record and show “authentic”. Closing this needs tag-side password protection (NTAG213 `PWD_AUTH`) or an originality-signature chip family (e.g. NTAG 21x DNA).
- **Single shared-secret trust:** both crypto paths still rely on one HMAC secret (`SHARED_SECRET`) to prove a write came from the real Pi. If that secret ever leaks, writes can be forged under either crypto path. Ed25519 device-signing (private key never leaves the Pi) would remove this single point of failure but is genuinely extra scope.
- **MITM tests prove integrity, not confidentiality:** the attack-sim suite shows tampering is caught even without TLS, but a LAN eavesdropper can still observe traffic metadata on plain HTTP today. This is exactly why HTTPS deployment (below) still matters and isn't replaced by these results.

4. What's Left To Do

4.1 Required before you can run or demo this

No Postgres server was reachable in the environment this work was done in, so **none of the new code has been run end-to-end against a live database yet**. Everything was verified as far as possible without one (module imports, route registration, the AES-GCM roundtrip, the paper-cipher regression test, NDEF byte-budget math, static file serving) — but the actual write → verify flow needs a real run.

- Set a real `DATABASE_URL` in `backend/.env` (currently a placeholder).
- Run `python init_db.py` then `python migrate.py`.
- Start the server: `python app.py`.
- In another terminal: `python -m pytest tests/ -v`, then `python tests/report.py` to generate the evidence report.
- Open `http://localhost:5000` in a browser and manually test the verify flow (authentic, expired, tampered, unknown cases).
- Test the Pi script against real hardware (PN532 + a physical NTAG213 tag) — the NDEF write path in particular has only been checked by pure byte-math, not against a real chip.

4.2 Phase 2 — deployment (explicitly not part of this round)

- Swap the Flask dev server (`app.run()`) for a production WSGI server (waitress or gunicorn).
- Put it behind a Cloudflare Tunnel (or similar) for a real HTTPS URL, matching your original plan.
- Update `BACKEND_URL` and `FRONTEND_VERIFY_BASE_URL` in both `.env` files to the deployed HTTPS domain.
- Revisit the rate limiter's storage backend if you ever move to multiple worker processes (the current in-memory store is per-process).

4.3 Optional further hardening (not blocking, worth documenting either way)

- NTAG213 `PWD_AUTH` page-password protection, to prevent the NDEF record on a genuine tag from being physically overwritten.

- Ed25519 device-signing on the Pi, to remove the single-shared-secret trust model.

4.4 Research-paper extras (from your original plan, still open)

- Benchmarking Ed25519 vs. the current ECDSA/paper-cipher approach for the paper's numbers.
- Running the NIST SP 800-22 randomness test suite against the nonce generator.

5. File Map

New or changed files in this round of work.

File	Status	What it is
backend/crypto_paper.py	New	Untouched paper cipher, moved out of utils.py
backend/crypto_modern.py	New	AES-256-GCM + HKDF production crypto
backend/admin_auth.py	New	X-Admin-Key decorator for admin routes
backend/audit.py	New	Writes to the new audit_log table
backend/migrate.py	New	Additive, idempotent schema migration
backend/tests/*.py	New	Functional + replay/clone/bruteforce/mitm attack-sim suite
frontend/*	New	Consumer verify webpage (HTML/CSS/JS, no build step)
pi/pi_app.py	New	Promoted, hardened Pi script (was backend/Pasted text.txt)
pi/requirements.txt	New	Hardware-specific Pi dependencies
backend/app.py	Changed	New verify/admin routes, nonce enforcement, rate limits, audit calls
backend/config.py	Changed	Loads AES_MASTER_KEY, ADMIN_API_KEY, FRONTEND_VERIFY_BASE_URL
backend/db.py	Changed	Fixed Windows console UnicodeEncodeError crash
backend/init_db.py	Changed	No longer drops the table on every run
backend/utils.py	Changed	Trimmed to generic helpers, re-exports paper cipher
backend/requirements.txt	Changed	Added flask-limiter, pytest
backend/Pasted text.txt	Deleted	Superseded by pi/pi_app.py
pi_app.py (root)	Deleted	Was empty/stray
requirements.txt (root)	Deleted	Corrupted duplicate of backend/requirements.txt

Generated by Claude as a working summary of the production-hardening changes made to this project. See git log for full commit history and diffs.